ENGINEERING GUIDE

VirtualGlove Engineering Toolkit

Repeatable analysis, camera, tracing, and native-research workflows.

2026 EDITION / IAIN BENNETT / MIT LICENSE

The VirtualGlove Engineering Toolkit is an optional, version-matched collection for people who want to measure, reproduce, or extend the camera-to-game pipeline. It is not required to install VirtualGlove, personalize a hand, adjust movement reach, or play a game.

The toolkit favours repeatable evidence over live guesswork. Most investigations begin with an existing trace or recorded clip, proceed through an output-free comparison, and reach a running Controller only after the offline result is understood.

Choose the right download

Download `VirtualGlove-Engineering-Tools.zip` from the same GitHub release as the Controller and RetroPie installations. Do not mix toolkit and device versions when comparing behaviour.

Also download `VirtualGlove-Engineering-Tools.zip.sha256`. Verify the download before extracting it:

```
SH
shasum -a 256 -c VirtualGlove-Engineering-Tools.zip.sha256
unzip VirtualGlove-Engineering-Tools.zip
cd VirtualGlove-Engineering-Tools
```

On Linux, `sha256sum -c` can be used instead of `shasum -a 256 -c`.

First self-check

The initial check needs only Python and does not open a camera, contact a device, install packages, or send controller input:

```
SH
python3 scripts/check-engineering-toolkit.py
```

It verifies the package manifest, paths, Python syntax, direct helper files, and every supported command's side-effect-free help interface. Resolve a failed self-check before collecting evidence.

Create the local environment

Python 3.10 or newer can run offline analysis and video review. The setup script creates `.venv-engineering` inside the extracted toolkit and records the exact resolved package versions in `.venv-engineering/virtualglove-engineering-environment.json`:

```
SH
python3 scripts/setup-engineering-tools.py
```

Activate it on macOS or Linux:

```
SH
. .venv-engineering/bin/activate
```

MediaPipe comparisons use Python 3.12, matching the supported Controller toolchain:

```sh
python3.12 scripts/setup-engineering-tools.py --with-mediapipe
```

Rerunning either setup is safe. Verify an existing environment without changing it by adding `--check` and the same MediaPipe option originally used. The check compares the toolkit version, setup mode, Python version, exact resolved package record, and validated direct dependency versions.

The public toolkit does not carry the Controller's ARM64 MediaPipe wheel. A Controller installation already has its validated runtime; do not replace it with a generic workstation package. Use the packaged tools from a workstation for offline analysis and orchestration, or the existing deployed engineering copy on a development Controller when direct camera access is required.

What is included

The generated `engineering-tools.json` records the exact toolkit version, complete file inventory, and these categories.

Category	Purpose	Changes a running system?
Offline analysis	Examine latency, motion, direction response, palm anchors, and preprocessing evidence	No
Camera and recognition	Record a guided clip or compare camera delivery and MediaPipe lanes	May take exclusive ownership of the camera; never sends controller output
Live system tracing	Observe or temporarily trace the Controller, receiver, and native core	Some commands restart measured services; production is restored when stopped normally
Native and accelerator research	Exercise a supplied libretro core, ncnn sidecar, or isolated inference experiment	Does not install a core by itself; may compile local research binaries
Toolkit support	Set up and validate the extracted environment	Creates files only inside the toolkit environment

Shared `src/powerglove_vision`, `config`, and `native` source is included where the tools import or inspect it. The package also contains the project licence, third-party notices, and this guide.

The archive intentionally excludes ROMs, recorded video, trace results, credentials, pairing tokens, player settings, personal calibration, cached models, compiled emulator cores, and private device data.

Repository-only maintenance tools are excluded: release publishing, device deployment, firmware stamping, package construction, PDF and screenshot generation, and documentation audits.

Start with offline evidence

Use `--help` on every command before its first run. These examples demonstrate the safest common entry points.

Summarize a Controller motion trace:

```
SH
python scripts/analyze-motion-trace.py TRACE.json --output motion-summary.json
```

Join Controller and receiver latency traces. Add `--core` only when a matching native-core trace exists:

```
SH
python scripts/analyze-latency-trace.py \
  --controller controller.json \
  --receiver receiver.json \
  --output latency-summary.json
```

Prepare a local slow-motion video for reviewed frame annotation:

```
SH
python scripts/analyze-latency-video.py \
  --video recording.mov \
  --protocol smoke \
  --capture-fps 120 \
  --output-dir video-review
```

Replay a retained camera clip through the proven recognition lane:

```
SH
python scripts/benchmark-vision-replay.py \
  clip.avi --quick --output replay.json
```

Recorded clips and generated reports remain local unless the operator moves or uploads them. Review them for faces, rooms, hostnames, or other private material before sharing.

Read-only live observation

The status sampler observes the public Controller status endpoint without opening the camera or changing controller delivery:

```
SH
python scripts/measure-vision-status.py \
  --status-url http://CONTROLLER.local:8088/status?statistics=1 \
  --seconds 60 \
  --phase movement \
  --output status-sample.json
```

The end-to-end preflight checks a development checkout and both devices. It uses SSH but does not change them:

```
SH
python scripts/prepare-end-to-end-session.py \
  --controller-status http://CONTROLLER.local:8088/status?statistics=1 \
  --controller-ssh arduino@CONTROLLER.local \
  --retropie-ssh pi@RETROPIE.local \
  --output-dir preflight
```

Use dedicated SSH identities through the corresponding identity options. Do not place private keys inside the toolkit directory or an evidence bundle.

Guided camera capture

Stop normal vision before allowing an engineering tool to own the camera. Keep controller output stopped and ensure no game depends on the camera during the test.

The guided recorder provides a local live preview and user-paced cues:

```
SH
python scripts/guided-vision-benchmark.py \
  --camera auto \
  --protocol tracking \
  --output guided-tracking.avi
```

The page address is printed when the recorder starts. Finish or interrupt the recorder normally so it releases the camera. Restart normal vision and confirm the Dashboard preview before playing.

Camera pipeline and exposure tools require explicit acknowledgement flags. Those flags confirm that the operator has stopped the ordinary camera worker; they are not shortcuts around that safety step.

Tracing a running two-device system

`manage-latency-traces.py` temporarily recreates measured services with bounded diagnostic tracing. Use its `start` and `stop` subcommands as one matched pair. The stop action restores production before collecting finalized traces.

Before starting:

1. Confirm Controller and RetroPie SSH access.
2. Confirm the expected game and emulator are selected.
3. Stop if either device is already unhealthy.
4. Choose a short bounded duration.
5. Keep a separate terminal ready to run the stop command.

Do not publish raw trace bundles. Although controller packets contain no camera frames, reports can contain host information, paths, timings, and configuration details.

Native emulator research

`run-nestopia-powerglove-trace.py` loads a caller-supplied libretro core and ROM without installing either one. The toolkit never supplies commercial ROMs. Keep ROM and scratch paths outside the extracted toolkit, and remove scratch copies when the run finishes.

The ncnn and MediaPipe Tasks tools are retained for reproducible comparison; they are not production alternatives. A faster isolated layer or delegate is not sufficient evidence. Any proposed runtime must preserve recognition, continuity, ordering, thermal stability, and complete camera-to-game latency.

Evidence checklist

Every retained result should record:

- VirtualGlove release and toolkit version
- Controller and RetroPie build identifiers
- Camera model, reader, frame rate, buffer count, exposure, and gain
- Python and resolved package environment record
- Exact command and input-file digests
- Whether Dashboard preview and statistics were open
- Test duration, operator cues, limitations, and unexpected interruptions
- Aggregate output filename and SHA-256 digest

Never treat results from different clips or physical movements as a controlled A/B comparison. Never combine latency percentiles measured on clocks that were not synchronized.

Cleanup and recovery

Generated environments, clips, traces, and reports are not managed by the VirtualGlove installer.

- Delete and recreate `.venv-engineering` when changing Python families.
- Keep one known-good environment record with important evidence.
- Move valuable clips and aggregate reports into a clearly named private evidence directory.
- Remove abandoned scratch directories and duplicate extracted toolkits.
- Do not clear the Controller's active `uv` cache merely to make space; it supports reliable offline worker restarts.
- Archive old development-device evidence before removing it.
- After any camera-owning experiment, confirm the Dashboard preview and normal game control before declaring recovery complete.

For active file locations and option definitions, see the online Configuration Reference. For the current runtime and device boundaries, see Architecture and flows. For why specific experiments were retained or rejected, see the Engineering Journey.